



SNAPSHOT

Two devices. Zero servers.

Contents

1. Executive Summary	3
2. Feature Catalogue	3
3. How It Works	5
4. Privacy & Security	6
5. Limitations & Honest Caveats	7
6. Quick Start Guide	8
7. Technical Specifications	9
8. Legal Notice & License	10
9. About the Author	10

This document describes SnapShare as deployed at <https://snapshare-share.vercel.app> on the date of issue. The underlying source code is open and tracked at <https://github.com/leonelferreirao373/snapshare>.

1. Executive Summary

SnapShare is a free, browser-based, peer-to-peer file transfer service. Two devices open the same URL; one shows a QR code or a six-character pairing code; the other scans or types it; the connection is established. Files then flow *directly between the two devices*, encrypted end-to-end, without ever passing through a SnapShare server.

There are **no accounts**. There is **no upload limit**. There is **no quality loss**. There is **no recurring cost** — for the operator or for the user. The service is hosted on a free Vercel static plan, relies on the public PeerJS signaling broker for the brief handshake, and is otherwise fully autonomous.

The product is **multi-device, multi-platform, and multi-language** (Portuguese and English, with Portuguese as the default). It includes light and dark theming, an installable Progressive Web App (PWA), drag-and-drop anywhere on a desktop screen, in-app QR scanning via camera or screen capture, automatic clipboard capture (text and images), tap-to-copy on received text payloads, file-type-aware iconography, resend, cancel, and per-peer disconnect.

ONE-SENTENCE PITCH

The fastest, most private way to move a file from one device to another, with nothing installed, no account created, and no third party touching the bytes.

2. Feature Catalogue

2.1 PAIRING

CAPABILITY	DETAIL
QR code	Always rendered in the header; tap to expand to a 220 px canvas. Encodes the live URL with the local peer identifier as a query parameter for instant auto-pairing.
Six-character code	Persistent across browser sessions (stored in <code>localStorage</code>). Formatted as <code>ABC-XYZ</code> ; auto-uppercase, auto-hyphen as the user types on the other device.
Copy to clipboard	One-tap copy of the formatted code with a 1.5-second confirmation state.
In-app QR scanner	Two sources: device camera (mobile) and screen-capture via <code>getDisplayMedia</code> (desktop). Decodes through the jsQR worker bundled at build time. Slides in from the right.
Multi-device topology	One device may pair with many others simultaneously. Files broadcast in parallel to every open peer.

CAPABILITY	DETAIL
One-tap reconnect	The most recent eight pairs are remembered. Each appears as a chip with a single button to resume the connection.

2.2 SENDING & RECEIVING

CAPABILITY	DETAIL
File source	System file picker, drag-and-drop into the drop zone, drag-and-drop anywhere on the page (desktop full-screen detection), or paste from clipboard (Ctrl+V / Cmd+V).
Multi-file	Pre-creates a row per file at the moment of selection. Each row updates its own progress percentage in real time.
Progress feedback	Updates approximately every one percent or sixty-four kilobytes, whichever is larger. Both sides see the bar advance.
Cancel	An × button appears on any in-progress row. Either side may cancel; the other side is notified and tears down its partial buffer.
Resend	Available on the sender's side once a transfer is complete. Re-broadcasts the same file to every currently paired peer — including those that joined <i>after</i> the original send.
Auto-save on receive	Binary payloads (images, video, archives) trigger an automatic download. On Android they land in /Download and are indexed by Gallery. On iOS Safari they land in the Files app.
Tap-to-copy on text	Text payloads under one megabyte are not auto-downloaded. The row becomes interactive and a tap copies the full content back to the system clipboard, with inline two-line preview.

2.3 CLIPBOARD

CAPABILITY	DETAIL
Auto-capture toggle	A clipboard button in the header (emerald when on). Captures the system clipboard whenever the SnapShare tab regains focus. New content becomes a transfer.
Supported types	Text (UTF-8) and any image type the browser exposes through <code>navigator.clipboard.read()</code> — PNG, JPEG, WebP, SVG, and similar.
Deduplication	A fingerprint of mime type, byte length, and the first 256 bytes is kept in memory. Identical re-reads are skipped.
Global paste	Independently of the toggle, Ctrl+V or Cmd+V anywhere on the page (outside form fields) extracts any file payloads and ships them.

2.4 QUALITY-OF-LIFE

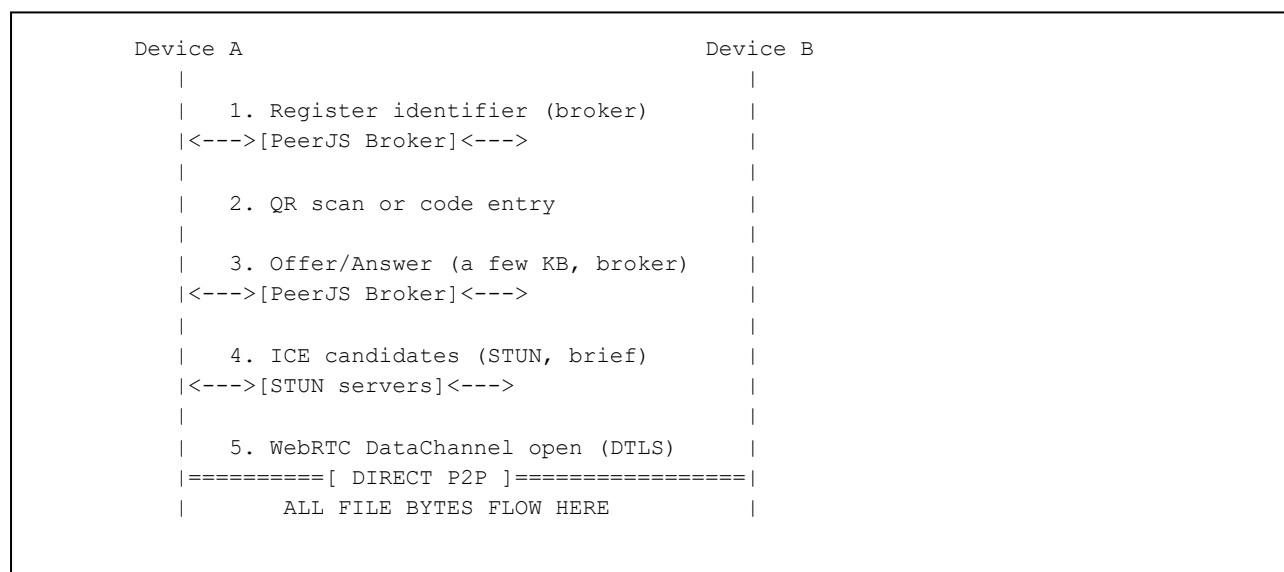
CAPABILITY	DETAIL
Light & dark theme	Toggle in the bottom-centre floating pill. Light mode uses a warm cream surface (<code>#f3f1ec</code>) chosen to reduce eye strain; cards lift with a subtle shadow.
Bilingual	Portuguese (default) and English. All UI strings are translated. Choice persists in <code>localStorage</code> .
Keep screen awake	Opt-in toggle holds a screen Wake Lock while paired, so a mobile tab is not suspended between transfers. Re-acquires on tab visibility change.
File-type icons	Around forty extensions mapped to representative icons (audio, video, image, code, archive, spreadsheet, document, generic).
Show all / show less	The transfers panel shows the six most recent rows by default; a toggle expands the full list.
Collapse all	Resets the visible expansion state and scrolls the page to the top.
Back to top	A floating arrow appears after roughly 400 pixels of vertical scroll; smooth-scrolls to the top.
PWA install	On supported browsers a footer button triggers the native install prompt. On iOS Safari an informational popover explains the Share → Add to Home Screen path. A standalone-mode badge indicates when the app is already installed.

3. How It Works

SnapShare is a single-page application written in TypeScript and React, bundled with Vite, and hosted as a fully static deployment on Vercel. All runtime logic executes inside the user's browser; the SnapShare backend consists of zero servers under operator control.

3.1 CONNECTION ESTABLISHMENT

- ① **Page load.** The browser registers a unique peer identifier with the PeerJS public broker over a brief WebSocket handshake. Identifiers are six alphanumeric characters, persisted in `localStorage` so they remain stable across visits.
- ② **Discovery.** Device A displays a QR code containing the URL with its peer identifier. Device B opens the same URL (typically by scanning the QR with the OS camera, or by typing the six-character code).
- ③ **Handshake.** Device B asks the broker to connect to Device A's identifier. The broker relays the WebRTC offer/answer between the two devices — a few kilobytes total. ICE candidates are exchanged using free public STUN servers (Google).
- ④ **Data channel.** A reliable WebRTC `DataChannel` opens directly between the two devices. From this point onwards, the broker is idle; files flow peer-to-peer.



3.2 FILE TRANSFER PROTOCOL

A file transfer is initiated by sending a `meta` JSON message containing a generated transfer identifier, the original filename, the declared MIME type, and the total byte length. The sender then streams the file in sixteen-kilobyte binary chunks. Backpressure is governed by the underlying `RTCDataChannel`: when the buffered amount exceeds sixteen megabytes the loop awaits the `bufferedamountlow` event before resuming. A terminating `eof` message closes the transfer; the receiver reassembles a `Blob` and either triggers an automatic download (binary) or activates tap-to-copy (text).

A complementary `cancel` message exists for either side to abort an in-flight transfer cleanly. A `hello` message exchanged on connection open carries platform metadata used for the friendly device labels.

3.3 WHAT LIVES WHERE

COMPONENT	HOSTED BY	COST
Static SPA bundle	Vercel (free static plan)	\$0
Signaling broker	PeerJS public broker	\$0
STUN servers	Google public STUN	\$0
TURN relay	Not configured	—
File storage	None — bytes flow peer-to-peer	—
Database	None	—

4. Privacy & Security

4.1 WHAT NEVER LEAVES THE TWO DEVICES

The contents of every file you transfer remain between the sending device and the receiving device. They are not uploaded to any server, do not pass through any cloud storage bucket, are never logged, and cannot be reviewed by the operator. There is no backend that could be subpoenaed or breached, because there is no backend that ever sees the payload.

4.2 WHAT DOES BRIEFLY TRAVERSE THE BROKER

To establish the peer-to-peer link, two short JSON messages — the WebRTC offer and answer — must be relayed between the two devices. These contain cryptographic fingerprints and network address candidates. They do *not* contain any user content. Once the link is open, the broker is no longer in the loop.

4.3 ENCRYPTION

WebRTC `DataChannel` connections are encrypted at the transport layer using **DTLS 1.2 or later**. The session keys are negotiated directly between the two devices. SnapShare does not implement an additional application-level cipher; it relies on the standardised, browser-enforced WebRTC security model, which is end-to-end by construction.

4.4 DATA RETENTION

SnapShare collects no analytics, sets no first-party tracking cookies, and does not store user data. The only persistent state is on the user's own device, in `localStorage`, and is limited to: the selected language, the chosen theme, the persistent peer identifier, the list of recently paired devices (up to eight), the clipboard-capture toggle state, and the keep-awake toggle state. The user may clear this state at any time through normal browser controls.

4.5 OPEN SOURCE

The complete source code is published under the MIT License at **github.com/leonelferreira0373/snapshare**. Independent verification of the claims in this section is therefore possible without relying on the operator's word.

5. Limitations & Honest Caveats

5.1 NETWORK REACH

Two devices on the same local network connect with one hundred percent reliability. Across different networks the success rate is roughly **ninety to ninety-five percent**, dependent on the NAT type at each end. The remaining cases — typically carrier-grade symmetric NAT combined with restrictive corporate firewalls — require a TURN relay, which SnapShare does not currently provision (paid

bandwidth). When a connection cannot be established, the device shows a clear error rather than retrying silently.

5.2 MOBILE BACKGROUND BEHAVIOUR

Mobile browsers suspend the JavaScript engine and tear down WebRTC connections when a tab loses foreground status. There is no portable browser-side mechanism to keep a peer connection alive in true background. SnapShare offers the *Keep screen awake* toggle, which holds a screen Wake Lock and keeps the tab considered visible while the screen is on; this is the maximum that can be achieved without a native application wrapper.

5.3 FILE SIZE

Received files are assembled in browser memory before being saved. Files above approximately five hundred megabytes are reliable on desktop browsers; on mobile, memory pressure may cause the tab to be killed. A streaming save-to-disk path using the File System Access API is on the roadmap but requires Chromium-class browsers.

5.4 CLIPBOARD REACH

Web browsers expose **images and text** from the system clipboard reliably. They do **not** expose arbitrary file types copied from the OS file manager. A video or document copied from Windows Explorer into the clipboard cannot be read by a web page; only its filename appears. For such payloads users should rely on drag-and-drop or the *Add files* button.

5.5 IOS DOWNLOAD PATH

On iOS Safari, downloads land in the system Files application rather than the Photos library. Users may save received images to Photos manually from within Files. This is a platform limitation.

6. Quick Start Guide

- 1 **Open the service.** Both devices visit <https://snapshare-share.vercel.app>. The first visit on a device generates and remembers a six-character pairing code.
- 2 **Pair.** On Device A, expand the QR code (chevron button beside the code). On Device B, scan it with the OS camera or the in-app scanner. Alternatively, type Device A's six-character code into Device B's pairing field — auto-capitalised and auto-hyphenated as you type.
- 3 **Confirm.** Both devices show *Paired devices (1)* with a green dot beside the partner's name.
- 4 **Send.** Drag a file onto the drop zone (or anywhere on the desktop page), tap *Add files*, paste with Ctrl+V, or — if the clipboard toggle is on — simply copy on your other application and return to the SnapShare tab.
- 5 **Receive.** The other side shows the incoming row with a live progress bar. On completion, the file downloads automatically; for text payloads, the row becomes tap-to-copy.
- 6 **Resend or cancel.** Completed sender rows offer *Reenviar*. In-progress rows offer the cancel cross. Either side may cancel; both sides observe the cancellation cleanly.

7. Technical Specifications

7.1 STACK

Language	TypeScript 5.6
UI framework	React 18 with hooks
Build tool	Vite 5.4
Styling	Tailwind CSS 3.4 (class-based dark mode)
Iconography	Lucide
P2P library	PeerJS 1.5
QR generation	qrcode.react 4.1
QR decoding	qr-scanner (web worker, jsQR-based)
Hosting	Vercel (static, free tier)
Bundle size	~100 KB JavaScript gzipped, ~6 KB CSS gzipped

7.2 BROWSER SUPPORT

BROWSER	STATUS
Chrome / Edge (desktop)	Full support including <code>clipboard.read()</code> for images
Chrome (Android)	Full support including Wake Lock and PWA install
Safari (macOS)	Full transfer support; clipboard image read varies by version
Safari (iOS 16.4+)	Transfer and Wake Lock supported; downloads land in the Files app
Firefox	Transfer supported; some clipboard rich types limited

7.3 TRANSFER CHUNKING

Chunk size	16,384 bytes (16 KB)
High water mark	16 MB buffered amount
Low water mark	1 MB buffered amount
Progress reporting	Every 1% or 64 KB, whichever is greater
Text payload cap for tap-to-copy	1 MB

8. Legal Notice & License

8.1 LICENSE

SnapShare is distributed under the **MIT License**. In plain terms: anyone may use, copy, modify, merge, publish, distribute, sublicense, or sell copies of the software, provided that the copyright notice and this license notice appear in all copies or substantial portions of the software.

8.2 DISCLAIMER OF WARRANTY

The software is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall the author or copyright holders be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the software or the use or other dealings in the software.

8.3 DATA AND CONTENT RESPONSIBILITY

SnapShare does not store, mediate, or moderate any content transferred between user devices. Users are solely responsible for the legality and appropriateness of files they choose to send. The operator has no technical means to monitor, intercept, retain, or recover transferred content.

8.4 THIRD-PARTY SERVICES

SnapShare relies on the PeerJS public broker for connection signalling and Google's public STUN servers for NAT discovery. Use of these services is governed by their respective terms. The operator of SnapShare has no contractual relationship with these providers and offers no service-level guarantee on their behalf.

8.5 COMPLIANCE

Because SnapShare does not collect, process, or store any personal data on behalf of users, no GDPR/CCPA controller-processor relationship arises out of standard use. The only data that lives at all is on the user's own device, in `localStorage`, under the user's exclusive control.

9. About the Author

LEONEL FERREIRA

Software engineer and operator based in **Luanda, Angola**. Founder of independent product work spanning e-commerce (*Geosstore*), urban mobility (*BAZA*), graphics tooling (*Klip*), productivity (*Lume*), and now peer-to-peer file transfer (*SnapShare*).

Portfolio • leonelferreira.vercel.app

Source code • github.com/leonelferreira0373/snapshare

Contact · leonelferreira0373@gmail.com

This document and the underlying software were authored by Leonel Ferreira. SnapShare is offered freely to the public in the interest of a more private and frictionless way to move files between devices. Bug reports, feature requests, and contributions are welcome through the GitHub repository above.

— END OF DOCUMENT —